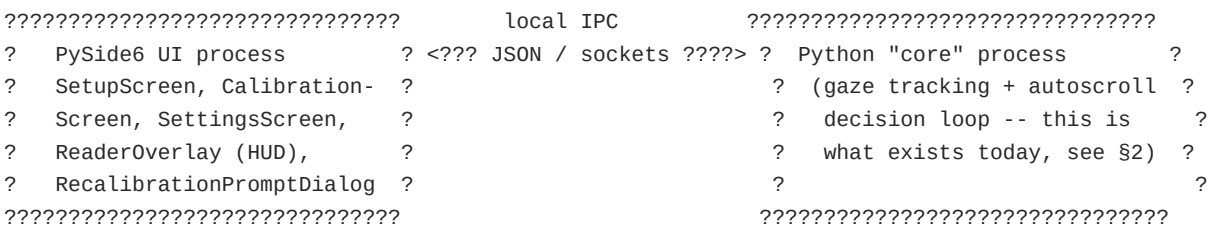
Doomscroll -- Lab 3 System Architecture

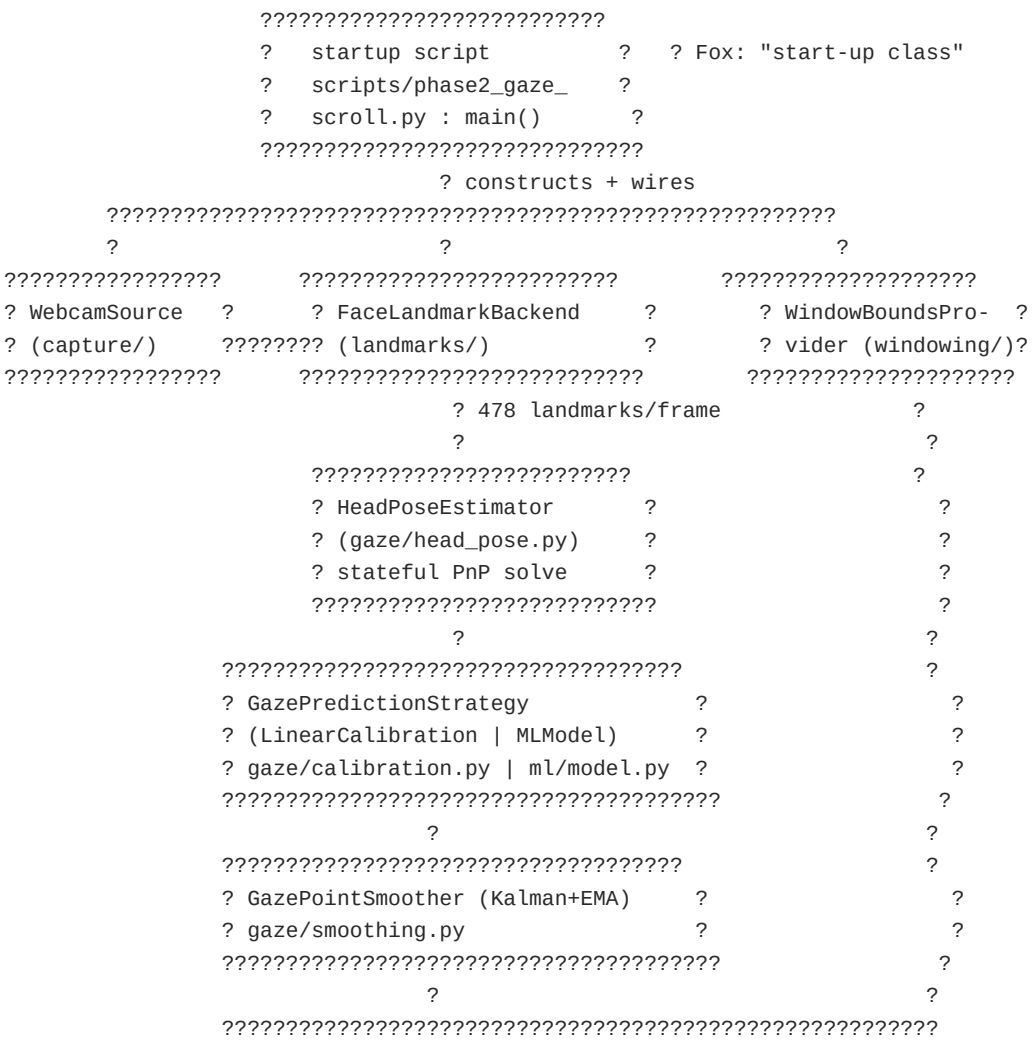
Per Lab 3 Section 3.1.2-3.1.3 (Fox heuristics, p.341-345). Two architectures are described: the **target architecture from Lab 2's tech stack discussion (not yet built), and the current** architecture that actually exists and runs today. Submitting the current, in-progress state is explicitly acceptable per the Lab 3 manual ("submission of an in-progress prototype... so long as the skeleton structure of the overall project is clear").

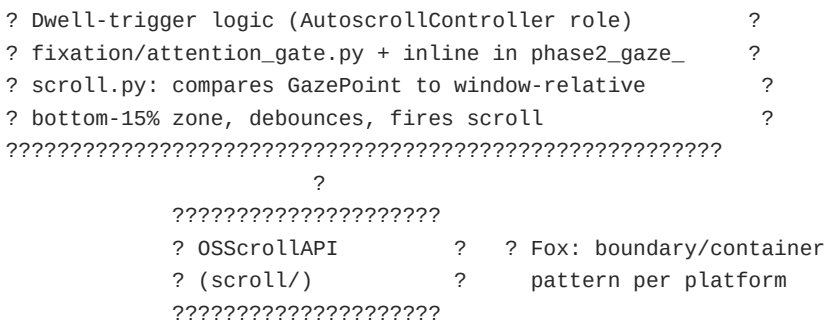
1. Target architecture (Lab 2 plan, still the Lab 4 goal)



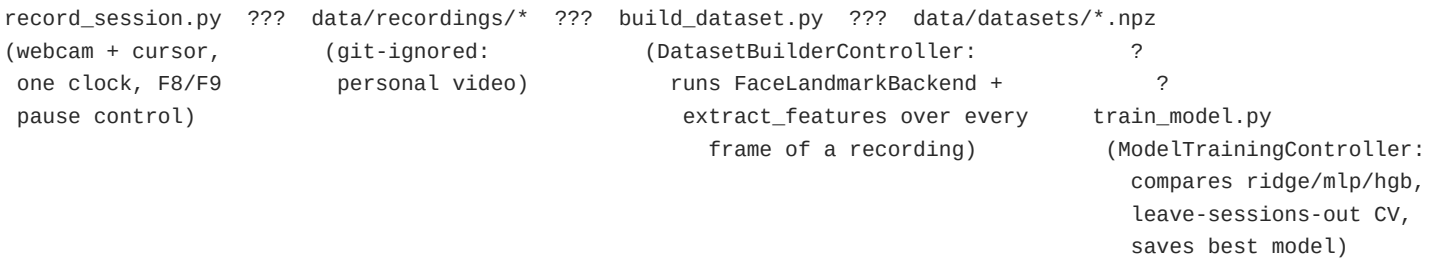
Single-process PySide6 (Qt widgets), per the team's Lab 2 opinion (Section 3.4.4 of tech_stack_report.pdf) -- not the AI's Tauri/sidecar recommendation, since the team already knows Python and the course timeline doesn't favor learning Rust. Whether this still holds is re-examined live in Section 3.3 below, since a lot has changed since Lab 2.

2. Current architecture (what's actually built and running)





Offline pipeline (separate from the live loop above, run manually by the team to produce models/gaze_model.joblib):



Fox heuristics checklist (Section 3.1.3)

- Start-up class -- scripts/phase2_gaze_scroll.py:main() for the live app; scripts/train_model.py:main() / scripts/build_dataset.py:main() for the offline pipeline. Each is a thin composition root: parses CLI args, constructs the concrete boundary objects (WebcamSource, FaceLandmarkBackend, WindowBoundsProvider), wires them into the control objects, and runs the loop -- no business logic lives in main() itself.
- Control classes distributing logic -- see the Strategy split in design_model.md Section 3: HeadPoseEstimator, the two GazePredictionStrategy implementations, and GazePointSmoother each own one responsibility and are composed, not merged into one class.
- Container classes -- SessionDataset/LabeledFrames hold per-session (N, FEATURE_DIM) numpy arrays (a Collection in the Fox/UML sense); CANDIDATES (ml/training.py) is a name->factory dict container for the three model types being compared.
- Design patterns -- Strategy (Section 3, design_model.md); the offline pipeline is a straightforward Pipeline/stage pattern (Section 4, design_model.md).

3. Why the architectures differ, and what that means for Lab 4

The live gaze-tracking/autoscroll loop (center of the diagram above) is a single Python process with no GUI -- it works, is tested live (manual dwell-trigger testing), and the debug overlay is cv2, not a packaged UI. The entire UI layer from Lab 2's design is unbuilt: no SetupScreen, no SettingsScreen, no persisted AutoscrollSettings. This is the single largest gap between plan and reality, and is the subject of the re-prompted AI tech-stack exercise in ai_tech_stack_reprompt.md -- worth deciding explicitly for Lab 4 rather than continuing to defer it.